

ECE3300L UART Project - Part 2

UART Receiver Core: 115200 Baud, 8-N-1

Project direction

Part 1 created the serial transmit path. In Part 2, you will add the matching receive path: synchronize an external RX signal, locate the center of each UART field, reconstruct one byte, and report whether the stop bit is valid. Board-level integration and application logic remain deferred.

What you will build

1. A synchronizer that safely brings the asynchronous RX signal into the 100 MHz clock domain.
2. A start-bit detector that recognizes the high-to-low transition beginning a UART frame.
3. A receive timer that first waits one-half bit period, then samples at one-bit intervals.
4. An 8-bit receiver that reconstructs D0 through D7 in least-significant-bit-first order.
5. Status outputs for receive activity, valid-byte completion, and an invalid stop bit.

UART configuration for Part 2

Setting	Required value	Meaning
Baud rate	115200 baud	The incoming line presents one serial bit every 1/115200 second.
Data length	8 bits	Each valid frame produces one received byte.
Parity	None	No parity field is expected after D7.
Stop bits	1	RX must be high when the stop field is sampled.
Bit order	LSB first	The first data sample is stored as D0.
Idle level	Logic 1	A falling transition from idle begins a candidate frame.

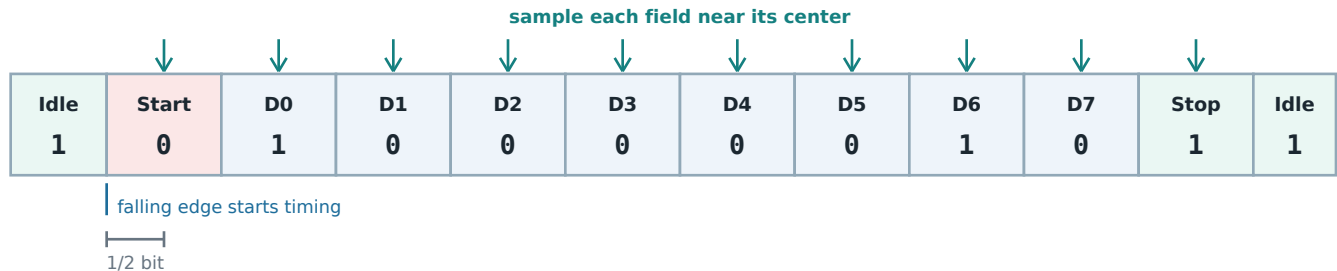
Lab objectives

- Understand why an asynchronous receiver samples near the center of each UART bit.
- Synchronize an external serial input before using it in synchronous control logic.
- Distinguish a candidate start edge from a valid start bit.
- Use timed samples and a finite-state machine to reconstruct a serial byte.
- Reject an invalid stop bit without overwriting the last valid output byte.

Receiver protocol and sampling

The receiver does not know when the transmitter changed each bit. It uses the falling start edge as a timing reference and samples away from field boundaries, where the data is most stable.

Receiver sample locations for ASCII 'A' = 0x41



After the synchronized falling edge, wait one-half bit period and verify that RX is still low. This rejects a short low pulse. From the center of the validated start bit, wait one full bit period for D0, then continue at one-bit intervals through D7 and STOP.

Receive-timing calculation

Required intervals

For a 100 MHz clock and 115200 baud, use **868 clocks per full bit** and **434 clocks per half bit**. A counter that starts at zero reaches the half-bit event at 433 and the full-bit event at 867.

Interval	Calculation	Approximate result	Use
System clock	1 / 100 MHz	10 ns	Clock all receiver registers.
Full UART bit	100,000,000 / 115,200	868 clocks	Space consecutive samples.
Half UART bit	868 / 2	434 clocks	Reach the center of START.
Complete frame	10 / 115200	86.8 us	Idle through the sampled stop field.

Receiver module interface

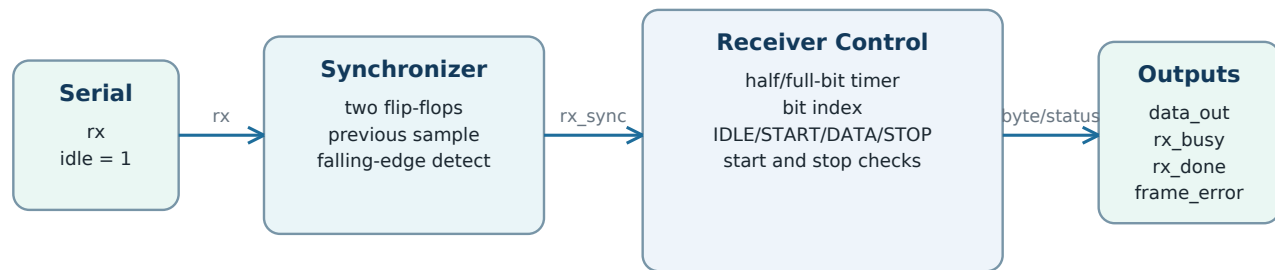
Signal	Direction	Purpose
clk	Input	100 MHz system clock used by all receiver logic.
rst_n	Input	Active-low reset; initializes the synchronized line to idle high.
rx	Input	Asynchronous serial input from the external UART source.
data_out[7:0]	Output	Most recently completed valid byte; holds its value between frames.
rx_busy	Output	High while validating or receiving a candidate frame.
rx_done	Output	One-system-clock pulse when a valid stop bit completes a byte.
frame_error	Output	One-system-clock pulse when the sampled stop bit is low.

Why the Part 1 tick is not used unchanged

The transmitter creates its own start boundary, but the receiver discovers an external boundary. RX timing must be restarted by the detected start edge and must use a half-bit interval before its first sample. A free-running or full-bit-only tick would not guarantee center sampling.

Module-level architecture

Part 2 adds one self-contained receive module. It uses the same 100 MHz clock and 115200-baud configuration as Part 1, but maintains receive timing internally.



The 100 MHz clock drives every register. No received signal is used as a clock.

An internal shift register collects D0 through D7; data_out changes only after a valid stop bit.

A later top module will connect RX to a board pin and route completed bytes to application logic.

Receiver state sequence

State	Timing	RX sample	Required behavior
IDLE	Indefinite	Watch continuously	Wait for a synchronized high-to-low transition; then assert rx_busy and clear the timer.
START	1/2 bit	Must be 0	Validate the candidate start bit. Return to IDLE if the line has already returned high.
DATA	8 full bits	D0 through D7	Sample one bit at each full-bit event and store it at the current bit index.
STOP	1 full bit	Must be 1	Accept the assembled byte if high; otherwise report a framing error.

Receive and status behavior

- Two serially connected flip-flops synchronize rx. A saved previous synchronized value allows the receiver to detect a falling transition.
- A falling edge starts a candidate frame, asserts rx_busy, and starts the half-bit timer.
- If RX is still low at the center of START, the frame is valid so far. If RX is high, treat the event as a false start and return to IDLE without producing a completion pulse.
- Sample D0 through D7 at successive full-bit intervals and place each sample into the matching position of an internal 8-bit register.
- At the stop sample, a high level copies the assembled byte to data_out and pulses rx_done. A low level leaves data_out unchanged and pulses frame_error.

Required design rules

- Use only the 100 MHz system clock in event controls. RX is data, never a clock.
- Reset both synchronizer stages and the saved previous sample to logic 1, matching the UART idle level.
- Make rx_done and frame_error low by default each clock so each assertion lasts exactly one cycle.
- Clear the timing counter whenever a new interval begins; do not let timing continue from an unrelated phase.
- Do not update data_out until the stop bit has been sampled high.

Scope boundary

Do not create a top module in this part. Pin constraints, loopback wiring, displays, FIFOs, command parsing, multi-byte messages, and integration with the Part 1 transmitter belong to later stages.

Step-by-step implementation

Implement the receiver in the order below. Keep the completed receive core independent of board-specific connections and later application logic.

1. Create `uart_rx.v` and define its interface

1. Declare the clock, active-low reset, asynchronous serial input, byte output, busy output, completion pulse, and framing-error pulse listed in the interface table.
2. Define parameters for a 100 MHz system clock and 115200 baud. Derive 868 clocks per bit and 434 clocks per half bit.
3. Define four receiver states: IDLE, START, DATA, and STOP.

2. Synchronize RX and identify a candidate start

1. Pass RX through two consecutive flip-flops clocked by the system clock. Use only the second synchronized value in the receiver state machine.
2. Save the preceding synchronized value. Detect the beginning of a candidate frame when the saved value is high and the current synchronized value is low.
3. Reset all synchronization and edge-history registers high so reset release does not create a false falling edge.

3. Add the receiver storage and timing registers

1. Add a timing counter wide enough for values 0 through 867, a 3-bit index for D0 through D7, and an 8-bit internal data register.
2. Reset the module to IDLE with `rx_busy`, `rx_done`, and `frame_error` low. Clear the timer, bit index, internal register, and `data_out`.
3. During normal operation, make both one-cycle status outputs low before executing state-specific behavior.

4. Receive and validate one UART frame

1. In IDLE, keep `rx_busy` low. On a falling edge, assert it, clear the timer and bit index, and enter START.
2. In START, wait 434 system-clock cycles. At the half-bit event, sample the synchronized RX value. Continue to DATA only if it remains low; otherwise cancel the candidate frame and return to IDLE.
3. In DATA, wait 868 clocks before each sample. Store the first sample as D0, then advance the index through D7. After storing D7, clear the timer and enter STOP.
4. In STOP, wait another 868 clocks and sample RX. If high, copy the internal byte to `data_out` and pulse `rx_done`. If low, preserve the old output byte and pulse `frame_error`.
5. After either stop-bit result, clear `rx_busy`, clear the timer, and return to IDLE.

5. Prepare the receive core for later expansion

1. Keep the system-clock and baud-rate values parameterized so later stages can change the serial rate without rewriting the control sequence.
2. Keep `data_out` stable between valid frames so a later controller has time to consume the byte.
3. Place only `uart_rx.v` in the new Part 2 design scope. Continue retaining the Part 1 transmitter files for later integration.

End of Part 2

At this point, the project specification contains independent single-byte transmit and receive cores using the same 115200-baud, 8-N-1 format. Board integration, buffering, and higher-level UART behavior remain outside this handout.